

in which they appear in the `cities` list. We can define an `itinerary` variable that will store this order in a list:

```
itinerary = list(range(0,N))
```

This is just another way of writing the following:

```
itinerary =  
[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,  
\30,31,32,33,34,35,36,37,38,39]
```

The order of the numbers in our `itinerary` is the order in which we're proposing to visit our cities: first city 0, then city 1, and so on.

Next, we'll need to judge this `itinerary` and decide whether it represents a good or at least acceptable solution to the TSP. Remember that the point of the TSP is to minimize the cost the salesman faces as he travels between cities. So what is the cost of travel? We can specify whatever cost function we want: maybe certain roads have more traffic than others, maybe there are rivers that are hard to cross, or maybe it's harder to travel north than east or vice versa. But let's start simply: let's say it costs one dollar to travel a distance of 1, no matter which direction and no matter which cities we're traveling between. We won't specify any distance units in this chapter because our algorithms work the same whether we're traveling miles or kilometers or light-years. In this case, minimizing the cost is the same as minimizing the distance traveled.

To determine the distance required by a particular `itinerary`, we need to define two new functions. First, we need a function that will generate a collection of lines that connect all of our points. After that, we need to sum up the distances represented by those lines. We can start by defining an empty list that we'll use to store information about our lines:

```
lines = []
```

Next, we can iterate over every city in our `itinerary`, at each step adding a new line to our `lines` collection that connects the current city and the city after it.

```
for j in range(0,len(itinerary) - 1):  
    lines.append([cities[itinerary[j]],cities[itinerary[j + 1]]])
```
